Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И УПРАВЛЕНИЕ» (ИУК)

КАФЕДРА «СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ» (ИУК3)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме

Студент группы ИУК3-71Б — А.А. Исаев

(подпись, дата) (И.О. Фамилия)

Руководитель курсового проекта — А.М. Макаренков

(подпись, дата) (И.О. Фамилия)

Калуга, 2026

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУК-3
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине _____
Студент группы ИУК3-71Б Исаев Артём Алексеевич
Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**
Направленность КП: **учебный**
Источник тематики: **кафедра ИУК-3**

Задание
– Описать модели типовых звеньев в пространстве состояний и способы перехода от структурной схемы к модели всей системы.
– Разработать алгоритм построения матриц A, B, C, D по структурной схеме, включая схемы с подсистемами и петлями без инерционных звеньев.
– Реализовать подсистему в серверной части системы моделирования и вывести полученную модель в веб-интерфейсе.
– Проверить подсистему на контуре с ПИ-регулятором, схеме с перекрёстными обратными связями и электрогидравлическом следящем приводе 8-го порядка независимым расчётом в MATLAB/Octave.

Оформление курсового проекта
Расчетно-пояснительная записка на _____ листах формата А4.
Перечень графического материала КП (плакаты, схемы, чертежи и т.п.):
– схема алгоритма построения модели – 1 лист формата А3;
– примеры построения модели и результаты проверки – 1 лист формата А3.

Дата выдачи задания «___» _____ 2026 г.

Руководитель _____ А.М. Макаренков
(подпись, дата) (И.О. Фамилия)

Студент _____ А.А. Исаев
(подпись, дата) (И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине _____

Студент группы ИУК3-71Б Исаев Артём Алексеевич

Тема курсового проекта: **Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме**

№	Наименование этапов	Срок, план	Срок, факт	Отметка руководителя	Куратор
1	Задание на выполнение КП	3-я нед.			
2	Изучение моделей звеньев и способов построения модели схемы	6-я нед.			
3	Разработка алгоритма и реализация подсистемы	10-я нед.			
4	Верификация на примерах с MATLAB/Octave	13-я нед.			
5	Оформление расчетно-пояснительной записки и графической части	15-я нед.			
6	Защита	17-я нед.			

Студент _____
(подпись, дата)

Руководитель _____
(подпись, дата)

Содержание

Перечень сокращений и обозначений.....6
Введение.....7
Постановка задачи.....8
1 Модель системы в пространстве состояний.....9
1.1 Что такое модель в пространстве состояний.....9
1.2 Модели типовых звеньев.....10
1.3 Как получить модель всей схемы.....11
2 Алгоритм построения модели.....13
2.1 Общий порядок работы.....13
2.2 Проверка схемы.....13
2.3 Раскрытие подсистем.....13
2.4 Объединение уравнений звеньев.....14
2.5 Таблицы соединений.....14
2.6 Итоговые матрицы и петли без инерционных звеньев.....14
3 Примеры построения модели.....16
3.1 Контур с ПИ-регулятором.....16
3.2 Перекрёстные обратные связи.....17
3.3 Электрогидравлический следящий привод 8-го порядка.....18
4 Программная реализация.....22
4.1 Место подсистемы в системе моделирования.....22
4.2 Входные данные.....22
4.3 Основные структуры данных.....23
4.4 Функции построения модели.....23
4.5 Что получает пользователь.....24
5 Верификация с MATLAB/Octave.....25
5.1 Как проверяется модель.....25
5.2 Результаты.....25
5.3 Переходные процессы.....26
6 Руководство пользователя.....29
6.1 Построение модели схемы.....29
6.2 Работа с подсистемами.....29
6.3 Если модель не построена.....30
Заключение.....31
Список литературы.....32

Приложение А. Главный алгоритм.....33
Приложение Б. Расчёт ЭГСП в MATLAB/Octave.....35

Перечень сокращений и обозначений

САУ – система автоматического управления

ПФ – передаточная функция

x, u, y – векторы состояния, входов и выходов

A, B, C, D – матрицы модели $\dot{x} = Ax + Bu,\ y = Cx + Du$

M, N – таблицы (матрицы) соединений схемы

κ – число обусловленности петли без инерционных звеньев

ЭГСП – электрогидравлический следящий привод

ЭМП – электромеханический преобразователь

ЭГУ – электрогидравлический усилитель

ГЦ – гидроцилиндр

API – программный интерфейс сервера

JSON – текстовый формат, в котором схема передаётся на сервер

Введение

Систему автоматического управления (САУ) обычно изображают структурной схемой: каждое звено — прямоугольник со своей передаточной функцией, стрелки показывают, куда идёт сигнал. Схема удобна человеку, но рассчитать по ней поведение системы напрямую нельзя: для расчёта нужны уравнения.

Программы моделирования решают эту задачу по-разному. Самый простой способ — считать блоки по очереди на каждом шаге времени: каждый блок по своему входу вычисляет свой выход и свою производную. Так работает, например, обычное моделирование в Simulink. Общих уравнений системы при этом не возникает, поэтому, чтобы узнать полюса, передаточную функцию или частотные характеристики всей схемы, требуются отдельные инструменты.

Другой способ — один раз превратить всю схему в одну модель вида «вход – состояние – выход» с матрицами A, B, C, D. По такой модели сразу получаются и переходные процессы, и полюса, и передаточная функция, и частотные характеристики. Но построить её по произвольной схеме — с несколькими контурами, подсистемами внутри подсистем и петлями, в которых сигнал проходит мгновенно, — непросто.

Курсовой проект выполнен как часть выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления». В проекте разработана подсистема, которая по структурной схеме, собранной пользователем в браузере, автоматически строит матрицы A, B, C, D всей системы и показывает пользователю, какой блок схемы отвечает за каждую переменную модели. Все остальные расчёты системы моделирования выполняются по этой модели.

Постановка задачи

В курсовом проекте требуется:

1. Описать, как типовые звенья записываются в пространстве состояний, и сравнить способы перехода от структурной схемы к модели всей системы.

2. Разработать алгоритм, который по схеме любой сложности строит матрицы A, B, C, D: схема может содержать обратные связи, подсистемы и петли без инерционных звеньев.

3. Предусмотреть понятные сообщения для схем, модель которых построить нельзя или ненадёжно.

4. Реализовать подсистему в серверной части (Python) и показать полученную модель во вкладке «Матрицы» веб-интерфейса.

5. Разобрать работу алгоритма на примерах: контур с ПИ-регулятором, перекрёстные обратные связи, электрогидравлический следящий привод 8-го порядка — и проверить полученные модели независимым расчётом в MATLAB/Octave.

1 Модель системы в пространстве состояний

1.1 Что такое модель в пространстве состояний

Любое линейное звено можно представить как устройство со входом u и выходом y (рисунок 1). Если звено инерционное, у него есть «память»: его выход зависит не только от входа в данный момент, но и от того, что было раньше. Например, интегратор накапливает входной сигнал, а апериодическое звено плавно догоняет вход. Величины, которые звено помнит, называют переменными состояния и обозначают x.



Рисунок 1 – Звено: вход u, состояние x, выход y

Модель в пространстве состояний — это два уравнения:

$$\dot{x} = A\,x + B\,u,\; y = C\,x + D\,u. \qquad (1)$$

Первое уравнение говорит, как быстро меняется состояние ($\dot{x}$ — производная по времени). Второе — как из состояния и входа получается выход. Матрицы — это просто таблицы коэффициентов:

– A показывает, как состояние влияет само на себя (например, затухание);

– B — как вход влияет на изменение состояния;

– C — какая комбинация переменных состояния видна на выходе;

– D — какая часть входа проходит на выход сразу, без задержки.

Если такая модель построена для всей схемы, из неё без дополнительных преобразований получаются основные характеристики системы [1, 2]:

– полюса системы — это собственные числа матрицы A, то есть корни уравнения $det(sI - A) = 0$; система устойчива, если у всех полюсов вещественная часть отрицательна;

– передаточная функция — $W(s)=C(sI-A)^{-1}B+D$;

– переходный процесс — решение первого уравнения при ступенчатом входе;

– частотная характеристика — $W(j\omega)$.

Поэтому модель всей схемы в пространстве состояний — основа для всех расчётов системы моделирования.

1.2 Модели типовых звеньев

Для каждого звена матрицы A_i, B_i, C_i, D_i выписываются по его параметрам один раз. Покажем это на апериодическом звене $W(s)=\frac{K}{Ts+1}$. Его уравнение во времени: $T\dot{y}+y=Ku$. Возьмём в качестве состояния сам выход, $x=y$. Тогда

$$\dot{x}=-\frac{1}{T}x+\frac{K}{T}u,\ y=x, \tag{2}$$

то есть $A=-1/T$, $B=K/T$, $C=1$, $D=0$. У интегратора $\frac{k}{s}$ состояние — накопленное значение: $A=0$, $B=k$, $C=1$, $D=0$. У усилителя и сумматора состояния нет совсем, выход появляется сразу: для усилителя $D=k$, для сумматора D — строка из $+1$ и -1 по знакам входов. Звено с произвольной передаточной функцией $\frac{N(s)}{D(s)}$ записывается в стандартной (канонической) форме [2]: порядок модели равен степени знаменателя. Все поддерживаемые звенья приведены в таблице 1.

Таблица 1 – Типовые звенья и их модели

Звено	**Передаточная функция**	**Состояний**	**Мгновенная передача**
Усилитель	k	0	да
Сумматор	$\pm u_1 \pm u_2 \pm \dots$	0	да
Интегратор	$\frac{k}{s}$	1	нет
Звено 1-го порядка	$\frac{K}{T s + 1}$	1	нет
Колебательное звено	$\frac{K \omega_0^2}{s^2 + 2\zeta \omega_0 s + \omega_0^2}$	2	нет
Передаточная функция	$\frac{N(s)}{D(s)}$	степень D(s)	если степени N и D равны
ФНЧ Баттерворта	порядок n, частота среза ω_c	n	нет
ПИД-регулятор	$K_p + \frac{K_i}{s} + \frac{K_d N s}{s + N}$	2	да

Последний столбец показывает, передаёт ли звено изменение входа на выход сразу ($D_i \neq 0$). Это свойство понадобится в разделе 2.6: если сигнал по петле проходит только через такие звенья, он обходит её мгновенно, и такую петлю приходится решать отдельно.

1.3 Как получить модель всей схемы

Можно выделить три способа (таблица 2).

Таблица 2 – Способы получить модель всей схемы

Способ	**Как работает**	**Ограничения**
Правила структурных преобразований (вручную)	Последовательные звенья перемножают, параллельные складывают, контур сворачивают по формуле $W/(1+W)$	Долго и легко ошибиться, если контуров несколько; даёт только передаточную функцию
Поблочный расчёт (как при обычном моделировании в Simulink)	На каждом шаге времени блоки считаются по очереди, по ходу сигнала	Нет общих матриц системы; не работает, если сигнал обходит петлю мгновенно
Построение единой модели (разработано в проекте)	Уравнения всех звеньев объединяются, соединения подставляются, получаются A, B, C, D	Только линейные звенья

Поблочный расчёт использовался в первой версии системы моделирования. Он прост, но не даёт матриц всей системы и не может

рассчитать схему, в которой сигнал проходит по петле без задержки. Поэтому в проекте разработан третий способ. Он не ищет на схеме контуры и не выбирает порядок их сворачивания: любое соединение для него — просто ещё одна запись в таблице соединений.

2 Алгоритм построения модели

2.1 Общий порядок работы

Алгоритм состоит из шести шагов; его схема и текст главной функции приведены в приложении А. Если на каком-либо шаге модель построить нельзя, пользователь получает сообщение с причиной и названиями блоков, которые нужно исправить.

2.2 Проверка схемы

Схема приходит на сервер в виде списка блоков и списка соединений «выход блока → вход блока» (формат JSON, пример — в разделе 4.2). Сначала проверяется, что схему вообще можно рассчитать: все блоки известного типа, параметры допустимы (например, постоянная времени $T>0$), к каждому входу подключён ровно один сигнал, нет соединений с несуществующими портами. Ошибки собираются списком, чтобы пользователь увидел их все сразу.

2.3 Раскрытие подсистем

Подсистема — это блок, внутри которого своя схема со своими входами и выходами. Для расчёта она не нужна как отдельный объект, поэтому алгоритм сначала «раскрывает» подсистемы: переносит их блоки на верхний уровень и соединяет связи, проходившие через границу подсистемы, напрямую. Вложенные подсистемы раскрываются так же, рекурсивно.

Чтобы имена блоков из разных подсистем не совпали, к имени каждого блока добавляется путь, например measurement::sensor_path::filter. Поэтому две копии одной подсистемы не влияют друг на друга, а пользователь потом видит, из какой подсистемы пришла каждая переменная модели. Если выход подсистемы соединён прямо с её входом, сигнал просто проходит насквозь.

2.4 Объединение уравнений звеньев

После раскрытия у каждого звена i есть свои матрицы A_i, B_i, C_i, D_i (раздел 1.2). Запишем уравнения всех звеньев вместе, не учитывая пока соединений. Состояния всех звеньев собираются в один вектор x, все входы — в вектор u_b, все выходы — в вектор y_b. Матрицы звеньев ставятся по диагонали общей матрицы, остальное заполняется нулями:

$$A_d = \begin{pmatrix} A_1 & 0 & \cdots \\ 0 & A_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}, \dot{x} = A_d x + B_d u_b,\ y_b = C_d x + D_d u_b. \tag{3}$$

Матрицы B_d, C_d, D_d составляются так же. Пока это просто «звенья, лежащие рядом».

2.5 Таблицы соединений

Теперь нужно учесть, как звенья соединены. Каждый вход получает ровно один сигнал: либо выход другого блока, либо внешнее воздействие r (ступенчатый вход). Это записывается двумя таблицами из нулей и единиц:

$$u_b = M\, y_b + N\, r. \tag{4}$$

В таблице M строки — входы блоков, столбцы — выходы блоков. Единица в строке «вход усилителя» и столбце «выход сумматора» означает, что выход сумматора подключён ко входу усилителя. Таблица N так же показывает, к каким входам подключены внешние воздействия. Аналогично записывается, что видят осциллографы: $y = S_y y_b + S_r r$. Таблицы составляются автоматически, проходом по списку соединений схемы.

2.6 Итоговые матрицы и петли без инерционных звеньев

Подставим соединения $u_b = M\, y_b + N\, r$ в уравнение выходов звеньев. Выходы оказываются связаны между собой:

$$\left(I - D_d M\right) y_b = C_d x + D_d N\, r, \tag{5}$$

где I — единичная матрица. Это обычная система линейных уравнений относительно y_b. Решив её, получаем выходы всех блоков через состояние и

воздействия: $y_b = Y_x x + Y_r r$. Подставив их в уравнения состояния и осциллографов, получаем модель всей схемы:

$$A = A_d + B_d M Y_x,\ B = B_d (M Y_r + N),\ C = S_y Y_x,\ D = S_y Y_r + S_r. \tag{6}$$

Решать всю систему уравнений целиком обычно не нужно. В большинстве схем сигнал идёт от блока к блоку, и выходы можно найти по очереди, подставляя уже найденные. Поэтому алгоритм сначала находит группы блоков, по которым сигнал может пройти по кругу мгновенно (через звенья с $D_i \neq 0$). Для этого используется поиск сильно связных компонент графа — алгоритм Тарьяна [5]. Остальные блоки считаются простой подстановкой по ходу сигнала, а систему уравнений приходится решать только внутри найденных групп.

Такая группа называется алгебраической петлёй. Её решение может быть очень чувствительным к коэффициентам. Простейший пример — усилитель k с положительной обратной связью:

$$y = k(u + y) \Rightarrow y = \frac{k}{1-k} u. \tag{7}$$

При $k = 1$ решения нет. При k, близком к единице, решение есть, но малая неточность в k сильно меняет результат: при $k = 0{,}999$ получаем $y = 999u$, а при $k = 0{,}998$ — уже $y = 499u$. Чувствительность петли оценивается числом обусловленности κ: оно показывает, во сколько раз относительная погрешность коэффициентов может вырасти в ответе [4]. Для петли из нескольких блоков G — матрица мгновенных передач внутри петли, σ_{min} — наименьшее сингулярное число:

$$\kappa = \frac{1 + \| G \|}{\sigma_{min}(I - G)},\ \text{для одного усилителя}\ \kappa = \frac{1 + |k|}{|1 - k|}. \tag{8}$$

Компьютер хранит числа примерно с 16 верными знаками. Если κ больше $1/\sqrt{\varepsilon} \approx 6{,}7 \cdot 10^7$, где $\varepsilon \approx 2{,}2 \cdot 10^{-16}$ — машинная точность, в ответе остаётся меньше половины верных знаков. В этом случае подсистема не выдаёт сомнительный результат, а сообщает, какие блоки образуют петлю, и предлагает изменить коэффициенты или добавить инерционное звено.

3 Примеры построения модели

Работа алгоритма показана на трёх схемах возрастающей сложности: контур с ПИ-регулятором, схема с перекрёстными обратными связями и электрогидравлический следящий привод (ЭГСП) 8-го порядка. Для первых двух модель построена вручную по шагам главы 2 и сравнена с ответом программы. Для всех трёх модель программы проверена в главе 5 независимым расчётом в MATLAB/Octave. Все три схемы есть в программе как готовые примеры.

3.1 Контур с ПИ-регулятором

Схема показана на рисунке 2: ПИ-регулятор $W_p(s) = K_п + K_и/s$ с $K_п = 2$, $K_и = 1$ управляет объектом из двух инерционных звеньев $\frac{1}{s+1}$ и $\frac{1}{0,5s+1}$; обратная связь единичная отрицательная.

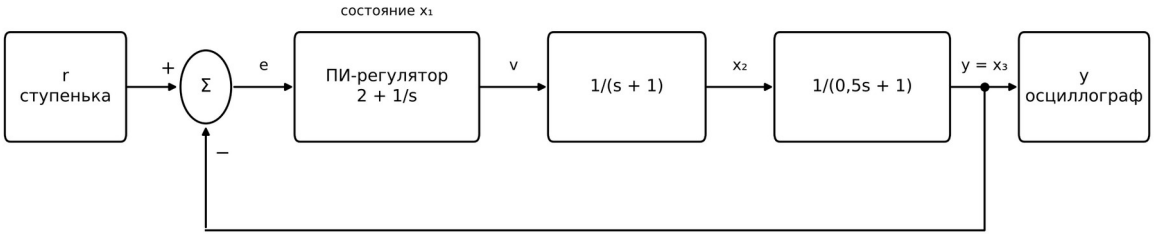


Рисунок 2 – Контур с ПИ-регулятором

Шаг 1. Уравнения блоков. У регулятора одно состояние x_1 — накопленная (интегральная) ошибка: $\dot{x}_1 = e$, выход $v = K_и x_1 + K_п e = x_1 + 2e$. Звенья (раздел 1.2):

$$\dot{x}_2 = -x_2 + u_2,\ \dot{x}_3 = -2x_3 + 2u_3,\ y = x_3. \tag{9}$$

Шаг 2. Соединения (таблица 3). Ступенька подключена ко входу «+» сумматора (таблица N).

Таблица 3 – Таблица соединений контура с ПИ-регулятором

Вход блока	**выход сумматора e**	**выход регулятора v**	**выход звена 1**	**выход звена 2**	**r (N)**
сумматор «+»	0	0	0	0	1
сумматор «−»	0	0	0	1	0
регулятор	1	0	0	0	0
звено 1	0	1	0	0	0
звено 2	0	0	1	0	0

Шаг 3. Подстановка по ходу сигнала:

$$e = r - x_3,\ v = x_1 + 2(r - x_3) = x_1 - 2x_3 + 2r, \tag{10}$$

$$\dot{x}_1 = -x_3 + r,\ \dot{x}_2 = x_1 - x_2 - 2x_3 + 2r,\ \dot{x}_3 = 2x_2 - 2x_3. \tag{11}$$

Шаг 4. Матрицы — коэффициенты при x_1, x_2, x_3 и r:

$$A = \begin{pmatrix} 0 & 0 & -1 \\ 1 & -1 & -2 \\ 0 & 2 & -2 \end{pmatrix},\ B = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix},\ C = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix},\ D = 0. \tag{12}$$

Программа выдала эти же матрицы. **Шаг 5. Проверка.** По правилам структурных преобразований $W = \dfrac{W_p W_1 W_2}{1 + W_p W_1 W_2} = \dfrac{4s+2}{s^3+3s^2+6s+2}$, а характеристический многочлен матрицы A равен $\det(sI - A) = s^3 + 3s^2 + 6s + 2$. Установившееся значение $W(0) = 2/2 = 1$: благодаря интегральной части регулятора статическая ошибка равна нулю.

3.2 Перекрёстные обратные связи

На рисунке 3 два местных контура перекрываются: обратная связь H_1 идёт с выхода G_2 на вход G_1, а H_2 — с выхода G_3 на вход G_2. Звено G_2 входит в оба контура, но ни один из них не вложен в другой. Поэтому правила последовательного, параллельного и встречного соединения напрямую не применяются: сначала нужно перенести точку съёма сигнала через звено, и только потом сворачивать. Для алгоритма программы эта схема ничем не сложнее предыдущей: каждая обратная связь — одна единица в таблице M.

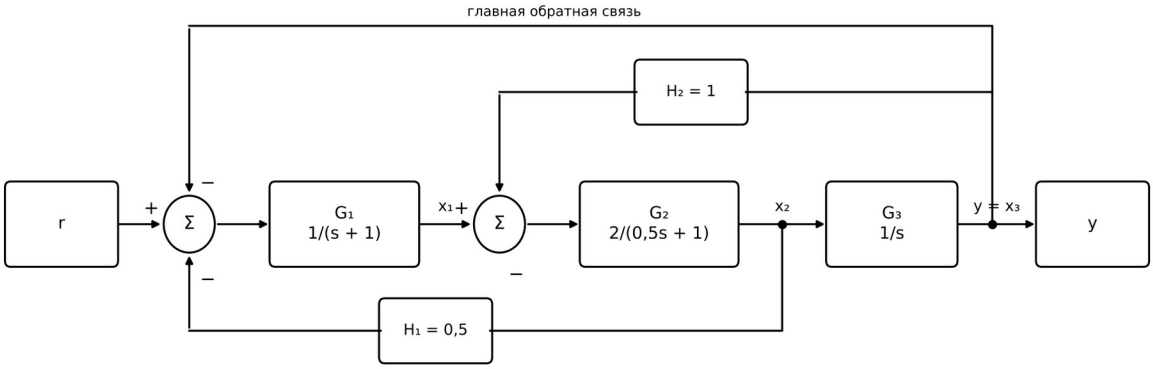


Рисунок 3 – Схема с перекрёстными обратными связями

Шаги 1–3. Уравнения звеньев и подстановка входов:

$$\dot{x}_1 = -x_1 + u_1,\ \dot{x}_2 = -2x_2 + 4u_2,\ \dot{x}_3 = u_3, \tag{13}$$

$$u_1 = r - x_3 - 0{,}5x_2,\ u_2 = x_1 - x_3,\ u_3 = x_2. \tag{14}$$

Шаг 4. Матрицы:

$$A = \begin{pmatrix} -1 & -0{,}5 & -1 \\ 4 & -2 & -4 \\ 0 & 1 & 0 \end{pmatrix},\ B = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix},\ C = (0 \quad 0 \quad 1),\ D = 0. \tag{15}$$

Шаг 5. Проверка по формуле Мейсона [1]. Прямой путь один: $P_1 = G_1G_2G_3$. Контуров три: $L_1 = -G_1G_2H_1$, $L_2 = -G_2G_3H_2$ и главный $L_3 = -G_1G_2G_3$. Все они попарно касаются (общее звено G_2), поэтому $\Delta = 1 - L_1 - L_2 - L_3$, $\Delta_1 = 1$:

$$W = \frac{G_1G_2G_3}{1 + G_1G_2H_1 + G_2G_3H_2 + G_1G_2G_3} = \frac{4}{s^3 + 3s^2 + 8s + 8}. \tag{16}$$

Характеристический многочлен матрицы A программы: $\det(sI - A) = s^3 + 3s^2 + 8s + 8$ — совпадает. Установившееся значение $W(0) = 0{,}5$.

3.3 Электрогидравлический следящий привод 8-го порядка

ЭГСП — привод, в котором малый электрический сигнал управляет мощным гидроцилиндром [8]. Напряжение U через усилитель подаётся на обмотку электромеханического преобразователя (ЭМП). Якорь ЭМП поворачивает заслонку, перепад давлений смещает золотник, и золотник открывает масло в гидроцилиндр. Поршень гидроцилиндра (ГД) движется, пока датчик обратной связи не скомпенсирует входное напряжение. Модель

взята из практических занятий по ЭГСП: уравнения (1.1)–(1.6) с допущениями $k_{Mp}=0$ и несжимаемостью жидкости в управляющих полостях:

$$L\dot{i}+R_a i=u,\ u=k_{yc}\left(U-k_{oc}Y\right), \tag{17}$$

$$J_я\ddot{\varphi}+h_я\dot{\varphi}+k_{M\varphi}\varphi=k_{Mi}i-k_{мос}x, \tag{18}$$

$$k_{Q_y p_y}p_y=k_{Q_y\varphi}\varphi-F_з\dot{x},\ m_з\ddot{x}+h_з\dot{x}+2C_{гд}x=F_з p_y, \tag{19}$$

$$\frac{V}{2E}\dot{p}+k_{Qp}p=k_{Qx}x-F\dot{Y},\ M\ddot{Y}+h\dot{Y}+CY=F\,p. \tag{20}$$

Значения параметров приведены в таблице 4.

Таблица 4 – Параметры ЭГСП

Параметр	**Значение**	**Параметр**	**Значение**
k_{yc}	50	$k_{Q_y\varphi}$	10^{-3} м³/с
L, R_a	0,4 Гн; 80 Ом	$k_{Q_y p_y}$	10^{-6} м⁴·с/кг
$J_я$	$0{,}3\cdot10^{-6}$ кг·м²	$F_з$	$0{,}785\cdot10^{-4}$ м²
$h_я$	$7\cdot10^{-5}$ Н·м·с	$m_з$, $h_з$	0,005 кг; 0,25 Н·с/м
$k_{M\varphi}$, k_{Mi}	0,1 Н·м; 1 Н·м/А	$C_{гд}$	1 Н/м
$k_{мос}$	10 Н	k_{Qx}	4,3 м²/с
V, E	$647\cdot10^{-6}$ м³; 10^7 Па	k_{Qp}	10^{-10} м⁴·с/кг
M, h, C	10 кг; 10 Н·с/м; 0	F	$1{,}99\cdot10^{-3}$ м²
k_{oc}	0,1 В/м	U	0,01 В (ступенька)

Структурная схема модели показана на рисунке 4. В ней четыре обратные связи, вложенные одна в другую: по скорости золотника ($F_з\dot{x}$), механическая ($k_{мос}x$), по скорости поршня ($F\dot{Y}$) и главная электрическая ($k_{oc}Y$).

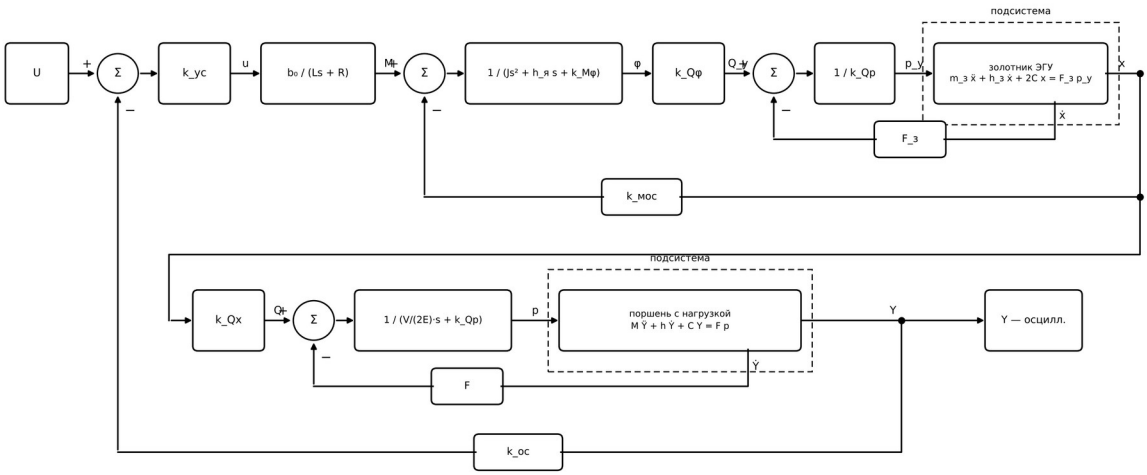


Рисунок 4 – Структурная схема ЭГСП

Как схема собрана в программе. В методичке скоростные обратные связи изображены звеньями $F_э s$ и $F s$ — идеальными дифференциаторами. Такое звено нельзя записать в пространстве состояний: степень числителя больше степени знаменателя. Но скорость $\dot{x}$ — это не новый сигнал, а переменная, которая уже есть внутри модели золотника. Поэтому золотник и поршень собраны как подсистемы из сумматора, усилителей и двух интеграторов (ускорение → скорость → положение), и у каждой подсистемы два выхода: положение и скорость (рисунок 5). Остальные звенья заданы передаточными функциями. На верхнем уровне 19 блоков, после раскрытия подсистем — 31.

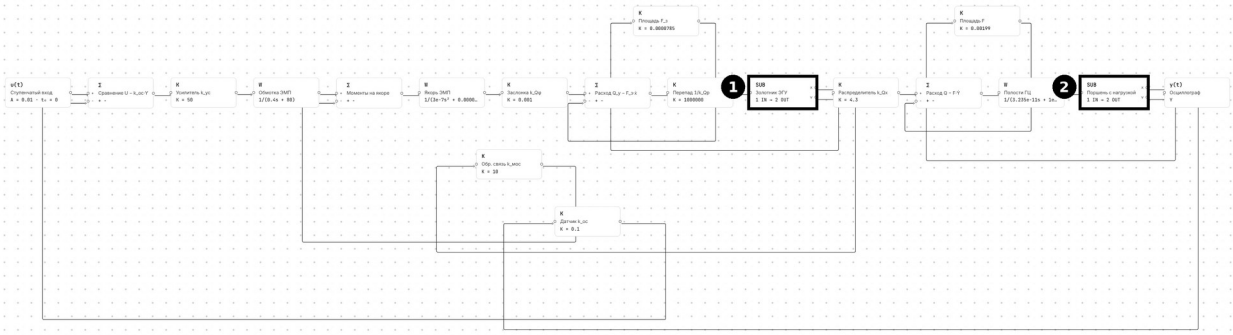


Рисунок 5 – ЭГСП в программе: (1) подсистема «Золотник ЭГУ», (2) подсистема «Поршень с нагрузкой»

Программа построила модель 8-го порядка; её переменные состояния — в таблице 5.

Таблица 5 – Переменные состояния модели ЭГСП

x	Блок	Физический смысл
x_1	обмотка ЭМП	ток i (с точностью до множителя)
x_2, x_3	якорь ЭМП	скорость и угол поворота якоря (каноническая форма звена 2-го порядка)
x_4	золотник :: скорость	скорость золотника ẋ
x_5	золотник :: положение	перемещение золотника x
x_6	полости ГЦ	перепад давлений p (с точностью до множителя)
x_7	поршень :: скорость	скорость поршня Ẏ
x_8	поршень :: положение	перемещение поршня Y — выход

Матрица A приведена в таблице 6. Её структура читается по схеме: например, -5 в первой строке — главная обратная связь ($k_{yc}k_{oc}/L$), -10 во

второй строке — механическая обратная связь $k_{мос}$, единицы под главной диагональю связывают скорость и положение. $B=[500\ldots0)^T$, $C=[0\ldots01)$, $D=0$.

Таблица 6 – Матрица A модели ЭГСП

	x₁	**x₂**	**x₃**	**x₄**	**x₅**	**x₆**	**x₇**	**x₈**
x_1	−200	0	0	0	0	0	0	−5
x_2	2,5	−233,3	$-3{,}333\cdot10^5$	0	−10	0	0	0
x_3	0	1	0	0	0	0	0	0
x_4	0	0	$5{,}233\cdot10^7$	−51,23	−400	0	0	0
x_5	0	0	0	1	0	0	0	0
x_6	0	0	0	0	4,3	−3,091	−0,00199	0
x_7	0	0	0	0	0	$6{,}151\cdot10^6$	−1	0
x_8	0	0	0	0	0	0	1	0

Характеристический многочлен матрицы, нормированный к $a_8=1$, и полюса системы:

$$s^8+488{,}66\,s^7+4{,}168\cdot10^5 s^6+9{,}389\cdot10^7 s^5+9{,}374\cdot10^9 s^4+\ldots \quad (21)$$

$$\ldots+1{,}205\cdot10^{12} s^3+5{,}063\cdot10^{13} s^2+1{,}608\cdot10^{15} s+1{,}730\cdot10^{16}, \quad (22)$$

$$s_{1,2}=-117{,}1\pm564{,}1\,j,\ s_3=-199{,}8,\ s_{4,5}=-2{,}88\pm110{,}8\,j, \quad (23)$$

$$s_{6,7}=-16{,}3\pm32{,}4\,j,\ s_8=-16{,}2. \quad (24)$$

Все полюса левее мнимой оси — привод устойчив. Числитель передаточной функции $1{,}730\cdot10^{17}$, поэтому $W(0)=10=1/k_{oc}$: при $U=0{,}01$ В поршень приходит в $Y=0{,}1$ м. Вывести эти коэффициенты вручную трудоёмко — в методичке это сделано символьно в MATLAB. В главе 5 они сравниваются с независимым расчётом в MATLAB/Octave (расхождение коэффициентов $2{,}9\cdot10^{-14}$).

4 Программная реализация

4.1 Место подсистемы в системе моделирования

Система моделирования состоит из веб-интерфейса (React, TypeScript) и сервера (Python, FastAPI). Пользователь собирает схему в браузере и нажимает «Запустить». Интерфейс отправляет схему на сервер, сервер строит модель, рассчитывает по ней графики и характеристики и возвращает результат. Разработанная подсистема — первый шаг этого расчёта: все остальные модули сервера работают уже с матрицами A, B, C, D. Файлы подсистемы перечислены в таблице 7.

Таблица 7 – Файлы подсистемы

Файл	Назначение
backend/app/validation/validator.py	проверка схемы (раздел 2.2)
backend/app/simulation/hierarchy.py	раскрытие подсистем (раздел 2.3)
backend/app/simulation/assembly.py	модели звеньев, таблицы соединений, петли без инерции, итоговые матрицы (разделы 2.4–2.6)
backend/app/simulation/model.py	общий порядок: проверка → раскрытие → построение модели
backend/app/analysis/system.py	полюса и передача модели в ответ сервера
frontend/src/components/simulation/ StateSpacePanel.tsx	вкладка «Матрицы» в интерфейсе
backend/scripts/kp_examples.py	схемы примеров главы 3
backend/verification/*.m	независимый расчёт примеров в MATLAB/Octave (глава 5)
backend/scripts/kp_verification.py	сравнение программы с MATLAB/Octave (глава 5)

4.2 Входные данные

Схема передаётся на сервер в формате JSON: список блоков с типом и параметрами и список соединений. Каждое соединение указывает блок и порт, откуда идёт сигнал (from), и блок и порт, куда он приходит (to). Ниже — схема примера 3.1 (координаты блоков на холсте опущены):

```
{
  "blocks": [
    {"id": "r", "type": "StepInput",
     "parameters": {"amplitude": 1.0, "t0": 0.0}},
    {"id": "e", "type": "Sum", "parameters": {"signs": ["+", "-"]}},
    {"id": "pi", "type": "PIDController",
     "parameters": {"kp": 2.0, "ki": 1.0, "kd": 0.0, "filter_n":
100.0}},
    {"id": "lag1", "type": "FirstOrderLag",
```

```
      "parameters": {"k": 1.0, "T": 1.0, "y0": 0.0}},
    {"id": "lag2", "type": "FirstOrderLag",
     "parameters": {"k": 1.0, "T": 0.5, "y0": 0.0}},
    {"id": "y", "type": "Scope", "parameters": {"label": "y"}}
  ],
  "connections": [
    {"from_block": "r", "from_port": "out",
     "to_block": "e", "to_port": "in1"},
    {"from_block": "e", "from_port": "out",
     "to_block": "pi", "to_port": "in"},
    {"from_block": "pi", "from_port": "out",
     "to_block": "lag1", "to_port": "in"},
    {"from_block": "lag1", "from_port": "out",
     "to_block": "lag2", "to_port": "in"},
    {"from_block": "lag2", "from_port": "out",
     "to_block": "e", "to_port": "in2"},
    {"from_block": "lag2", "from_port": "out",
     "to_block": "y", "to_port": "in"}
  ]
}
```

4.3 Основные структуры данных

Основные структуры данных подсистемы приведены в таблице 8.

Таблица 8 – Структуры данных

Структура	Что хранит
BlockRealization	матрицы одного звена A_i, B_i, C_i, D_i и начальное состояние
StateEntry	какому блоку принадлежит переменная состояния и её номер внутри блока
AlgebraicLoop	блоки петли без инерции и её число обусловленности κ
LinearModel	итоговые матрицы A, B, C, D, начальное состояние, список воздействий и осциллографов, найденные петли
ModelAssemblyError	список сообщений, если модель построить нельзя

4.4 Функции построения модели

Главная функция assemble_linear_model (листинг 1 приложения А) проходит по блокам раскрытой схемы, объединяет их матрицы, заполняет таблицы соединений по списку связей и вычисляет итоговые матрицы по формулам раздела 2.6. Модель отдельного звена по его типу и параметрам строит функция block_realization, решение уравнений выходов по группам блоков и проверку петель — функция _solve_algebraic_equations, раскрытие подсистем — функция _expand_level.

Для вычислений используются библиотеки NumPy и SciPy [6, 7]: из SciPy взяты поиск сильно связных компонент графа, LU-разложение для решения петель и перевод передаточной функции в каноническую форму.

4.5 Что получает пользователь

Построенная модель возвращается в ответе сервера вместе с результатами моделирования. В интерфейсе её показывает вкладка «Матрицы» (рисунок 6). В ней выводятся:

– матрицы A, B, C, D, у которых строки и столбцы подписаны именами блоков;

– таблица «Как собрана модель»: какому блоку и какой подсистеме принадлежит каждая переменная состояния;

– найденные петли без инерционных звеньев и их число обусловленности κ;

– размеры модели в строке состояния: n — число состояний, m — входов, p — выходов.

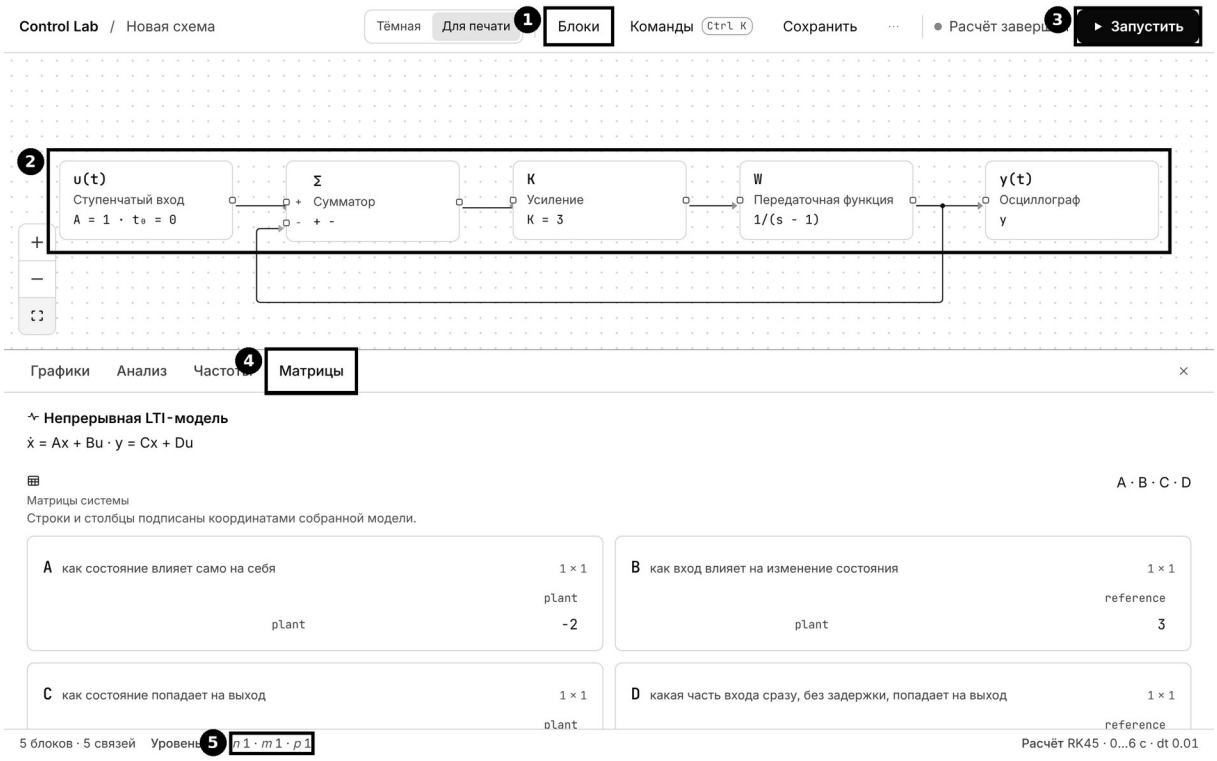


Рисунок 6 – Окно системы: (1) библиотека блоков, (2) схема, (3) запуск расчёта, (4) вкладка «Матрицы», (5) размеры модели

5 Верификация с MATLAB/Octave

5.1 Как проверяется модель

Чтобы убедиться, что подсистема строит модель правильно, каждая схема главы 3 рассчитана вторым, независимым способом — в MATLAB-совместимом пакете GNU Octave с пакетом control (функции tf, feedback, pole, lsim работают так же, как в MATLAB Control System Toolbox [9, 10]). Скрипты лежат в каталоге backend/verification и запускаются без изменений и в MATLAB.

– **Независимый путь (MATLAB/Octave).** Передаточная функция получается последовательным сворачиванием структурной схемы, как в методичке по ЭГСП (приложение 1 занятия 02), только численно. Для перекрёстных связей — с переносом точки съёма сигнала. Переходный процесс считается функцией lsim.

– **Путь программы.** Модель A, B, C, D строится по схеме разработанной подсистемой; передаточная функция вычисляется из матриц, полюса — как собственные числа A, а переходный процесс — тем же путём, что и по кнопке «Запустить» (метод RK45).

– **Сравнение:** коэффициенты передаточной функции (знаменатель нормирован к старшему коэффициенту 1), полюса и график $y(t)$ на одной и той же сетке времени.

Сравнение запускается одной командой (если Octave установлен, скрипты .m выполняются заново):

```
cd backend
python scripts/kp_verification.py
```

Скрипт для ЭГСП приведён в приложении Б.

5.2 Результаты

Результаты сравнения приведены в таблице 9.

Таблица 9 – Наибольшие относительные расхождения программы и MATLAB/Octave

Схема	n	Знаменатель	Числитель	Полюса	y(t)
Контур с ПИ-регулятором	3	$7{,}8\cdot10^{-16}$	$7{,}8\cdot10^{-16}$	$2{,}2\cdot10^{-16}$	$5{,}8\cdot10^{-9}$
Перекрёстные обратные связи	3	$8{,}9\cdot10^{-16}$	$2{,}2\cdot10^{-15}$	$1{,}3\cdot10^{-15}$	$2{,}3\cdot10^{-9}$
ЭГСП 8-го порядка	8	$2{,}9\cdot10^{-14}$	$1{,}0\cdot10^{-14}$	$4{,}7\cdot10^{-14}$	$7{,}0\cdot10^{-9}$

Коэффициенты передаточных функций и полюса совпадают до 13–15 значащих цифр — это предел точности чисел с плавающей запятой. Значит, модель программы и модель MATLAB/Octave — одна и та же система. Графики $y(t)$ отличаются меньше чем на 10^{-8} от размаха сигнала: это заданная точность численного интегрирования RK45 (10^{-8}), а не ошибка модели. Коэффициенты ЭГСП приведены в таблице 10.

Таблица 10 – ЭГСП: коэффициенты знаменателя передаточной функции

Коэффициент	Программа	MATLAB/Octave
a_8	1	1
a_7	488,6569734	488,6569734
a_6	$4{,}168276922\cdot10^{5}$	$4{,}168276922\cdot10^{5}$
a_5	$9{,}388871922\cdot10^{7}$	$9{,}388871922\cdot10^{7}$
a_4	$9{,}373583697\cdot10^{9}$	$9{,}373583697\cdot10^{9}$
a_3	$1{,}204873189\cdot10^{12}$	$1{,}204873189\cdot10^{12}$
a_2	$5{,}062753131\cdot10^{13}$	$5{,}062753131\cdot10^{13}$
a_1	$1{,}608112725\cdot10^{15}$	$1{,}608112725\cdot10^{15}$
a_0	$1{,}730356775\cdot10^{16}$	$1{,}730356775\cdot10^{16}$

5.3 Переходные процессы

На рисунках ниже переходный процесс, рассчитанный программой (сплошная линия, так его показывает осциллограф во вкладке «Графики»), наложен на результат MATLAB/Octave (кружки).

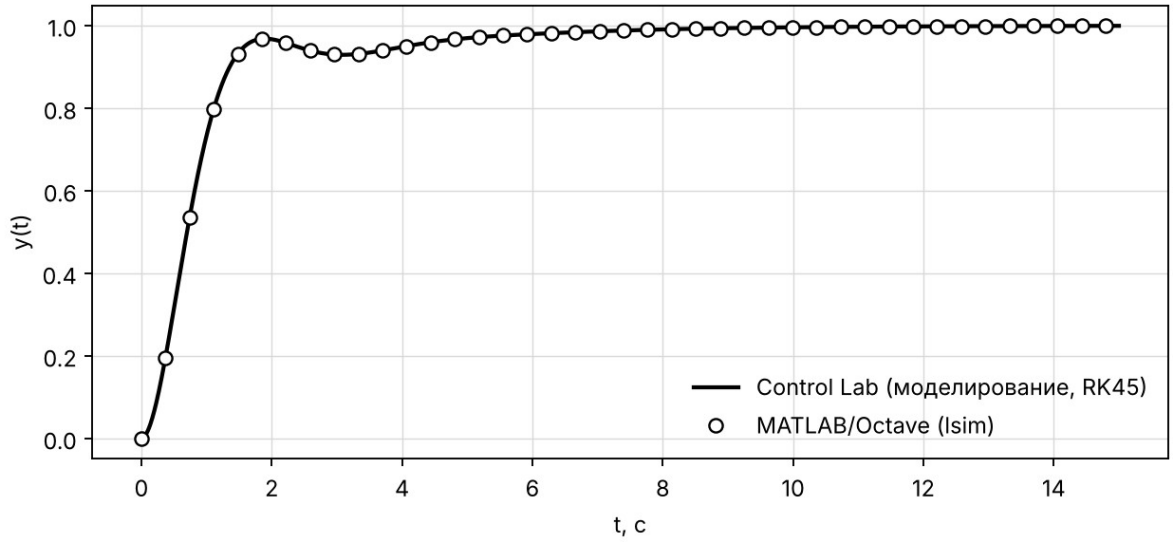


Рисунок 7 – Контур с ПИ-регулятором: программа и MATLAB/Octave

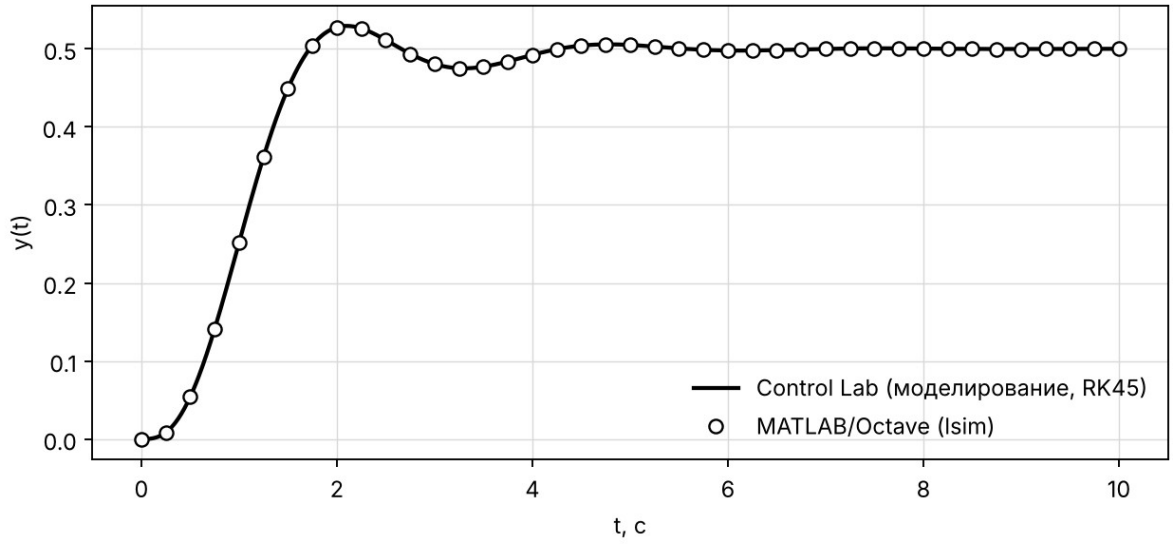


Рисунок 8 – Перекрёстные обратные связи: программа и MATLAB/Octave

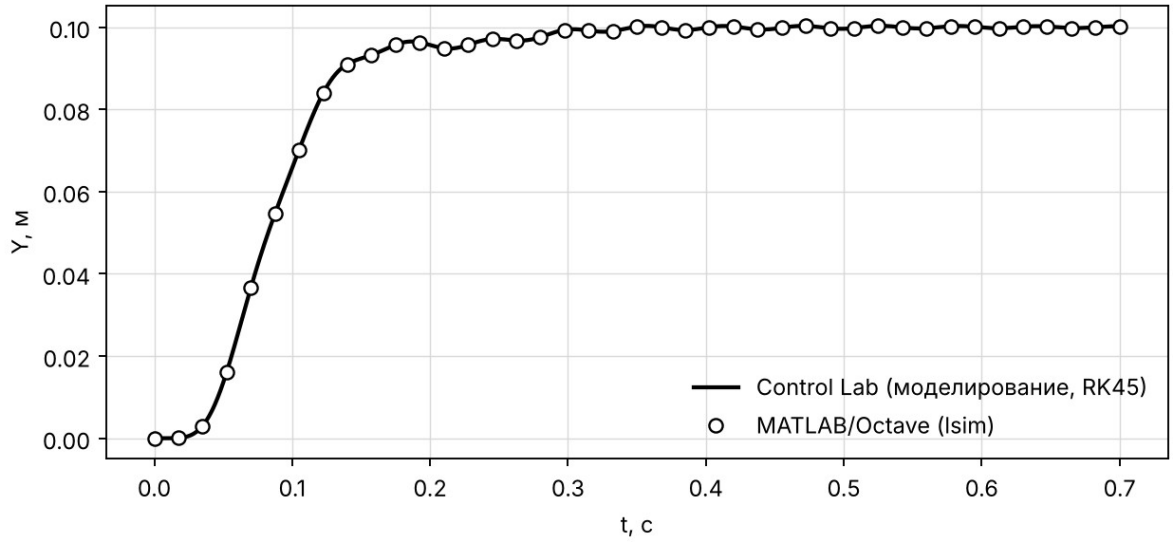


Рисунок 9 – ЭГСП, U = 0,01 В, интервал 0…0,7 с: программа и MATLAB/Octave

Кривые совпадают на всём интервале. Для ЭГСП поршень за 0,3 с приходит в установившееся положение 0,1 м с небольшими затухающими колебаниями — их дают слабо задемпфированные полюса $-2,88 \pm 110,8\,j$.

6 Руководство пользователя

6.1 Построение модели схемы

Основные элементы окна отмечены цифрами на рисунке 6 (раздел 4.5).

1. Откройте систему моделирования в браузере. Выберите готовый пример на пустом холсте или соберите схему из блоков: кнопка «Блоки» (1) открывает библиотеку, блоки перетаскиваются на холст (2), выход блока соединяется со входом другого блока мышью.

2. Нажмите «Запустить» (3) или Ctrl+Enter.

3. Откройте вкладку «Матрицы» (4). В ней — матрицы A, B, C, D и таблица «Как собрана модель». Размеры модели выводятся в строке состояния (5).

4. Чтобы изменить параметр, щёлкните блок и введите новое значение в поле (1) на рисунке 10. Модель будет перестроена при следующем запуске.

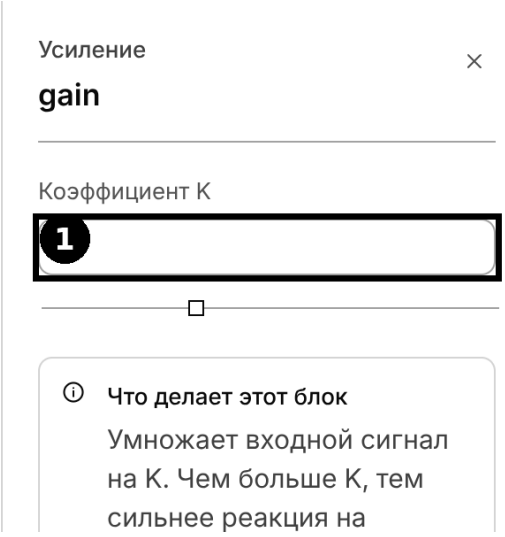


Рисунок 10 – Панель параметров блока: (1) — поле коэффициента

6.2 Работа с подсистемами

Подсистема на схеме отмечена надписью SUB (рисунок 11, отметка 1). Двойной щелчок открывает её содержимое (рисунок 12). После запуска таблица «Как собрана модель» показывает путь подсистемы для каждой переменной (рисунок 13).

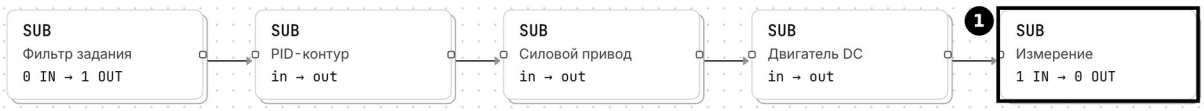


Рисунок 11 – Схема из подсистем: (1) — подсистема «Измерение»

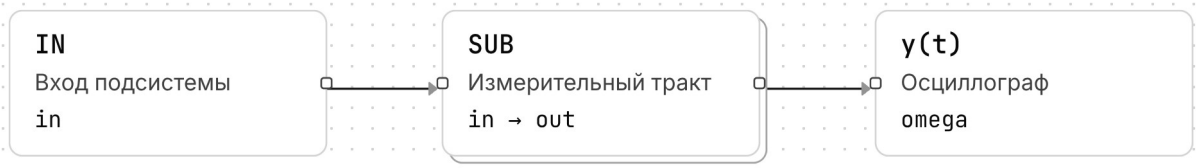


Рисунок 12 – Содержимое подсистемы «Измерение»

Как собрана модель

Координаты x связаны с динамическими блоками исходной схемы.

Координата	Состояние	Блок	Подсистема
x_1	**controller::pid.x1**	controller::pidPIDController	controller
x_2	**controller::pid.x2**	controller::pidPIDController	controller
x_3	**drive::drive_lag**	drive::drive_lagFirstOrderLag	drive
x_4	**motor::motor_model.x1**	motor::motor_modelSecondOrderOscillator	motor
x_5	**motor::motor_model.x2**	motor::motor_modelSecondOrderOscillator	motor
x_6	**measurement::sensor_path::adc_stage::signal_filter::filter**	measurement::sensor_path::adc_stage::signal_filter::filterFirstOrderLag	measurement / s

Рисунок 13 – Таблица «Как собрана модель» для схемы с подсистемами

6.3 Если модель не построена

Если модель построить нельзя, вместо результатов открывается панель «Проблемы» (рисунок 14). Каждое сообщение называет причину и блоки, которые нужно исправить. Чаще всего это неподключённый вход, недопустимый параметр или петля без инерционных звеньев. В последнем случае помогает изменить коэффициент петли или добавить в неё инерционное звено.

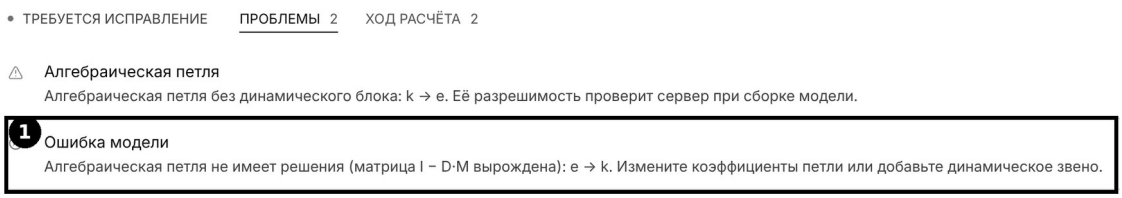


Рисунок 14 – Панель «Проблемы»: петля без инерционных звеньев, которую нельзя решить

Заключение

В ходе курсового проекта разработана подсистема построения математической модели САУ в пространстве состояний по структурной схеме для веб-ориентированной системы моделирования.

Разработан алгоритм, который по схеме любой сложности строит матрицы A, B, C, D всей системы. Схема проверяется, подсистемы раскрываются, уравнения звеньев объединяются, соединения записываются таблицами, а уравнения выходов решаются по ходу сигнала. Петли без инерционных звеньев решаются отдельно с оценкой надёжности; ненадёжные и нерешаемые петли не дают сомнительного результата, а сопровождаются понятным сообщением.

Подсистема реализована в серверной части, а полученная модель показывается во вкладке «Матрицы»: пользователь видит матрицы и то, какому блоку и какой подсистеме принадлежит каждая переменная.

Работа алгоритма разобрана на трёх схемах: контуре с ПИ-регулятором, схеме с перекрёстными обратными связями и электрогидравлическом следящем приводе 8-го порядка (31 блок после раскрытия подсистем). Для первых двух матрицы, полученные программой, совпали с ручным расчётом. Для всех трёх модель программы сверена с независимым расчётом в MATLAB/Octave: коэффициенты передаточных функций и полюса совпали с относительной погрешностью не более $4{,}7 \cdot 10^{-14}$, а переходные процессы — не более $7{,}0 \cdot 10^{-9}$ от размаха сигнала, что соответствует точности вычислений.

Разработанная подсистема является основой выпускной квалификационной работы: по построенной модели выполняются моделирование, анализ устойчивости и частотный анализ.

Список литературы

1. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.

2. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.

3. Ogata K. Modern Control Engineering. 5th ed. – Upper Saddle River: Prentice Hall, 2010. – 894 p.

4. Голуб Дж., Ван Лоун Ч. Матричные вычисления. – М.: Мир, 1999. – 548 с.

5. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М.: Вильямс, 2013. – 1328 с.

6. NumPy Documentation [Электронный ресурс]. – URL: https://numpy.org/doc/ (дата обращения: 01.10.2026).

7. SciPy API Reference [Электронный ресурс]. – URL: https://docs.scipy.org/doc/scipy/reference/ (дата обращения: 01.10.2026).

8. Моделирование и анализ электрогидравлического следящего привода: задания к практическим занятиям 01–03 [Электронный ресурс]. – Калуга: КФ МГТУ им. Н.Э. Баумана, кафедра ИУК3, 2026.

9. GNU Octave Documentation; Octave control package [Электронный ресурс]. – URL: https://docs.octave.org/latest/, https://gnu-octave.github.io/packages/control/ (дата обращения: 20.10.2026).

10. MATLAB Control System Toolbox Documentation: tf, feedback, lsim [Электронный ресурс]. – URL: https://www.mathworks.com/help/control/ (дата обращения: 20.10.2026).

Приложение А. Главный алгоритм

Схема алгоритма приведена на рисунке 15, его реализация — в листинге 1. Функция получает схему, в которой подсистемы уже раскрыты, и возвращает матрицы A, B, C, D.

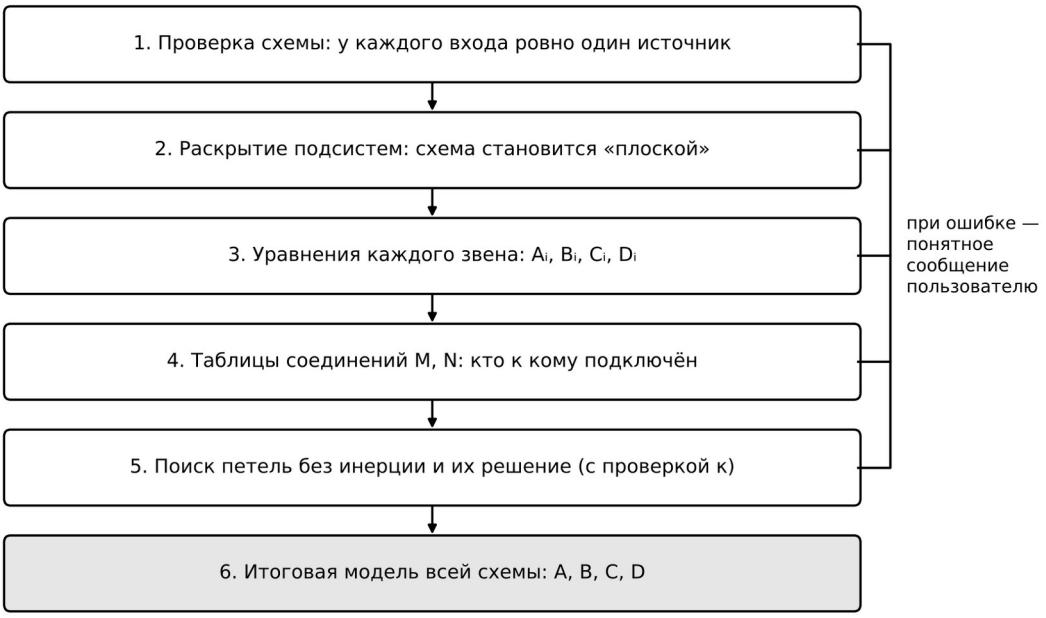


Рисунок 15 – Схема алгоритма построения модели

Листинг 1 – Построение модели схемы (backend/app/simulation/assembly.py)

```
def assemble_linear_model(flat: Diagram) -> LinearModel:
    """Assemble a validated, flattened diagram into (A, B, C, D)."""

    sources: list[StepSource] = []
    scopes: list[ScopeOutput] = []
    source_index: dict[str, int] = {}
    scope_index: dict[str, int] = {}

    dynamic: list[tuple[Block, BlockRealization]] = []
    output_index: dict[str, int] = {}      # block id -> row of y_b
    input_offset: dict[str, int] = {}      # block id -> first row of u_b
    states: list[StateEntry] = []
    total_inputs = 0

    for block in flat.blocks:
        if block.type in SOURCE_TYPES:
            source_index[block.id] = len(sources)
            sources.append(
                StepSource(
                    block.id,
                    get_numeric_parameter(block.parameters, "amplitude", 1.0),
                    get_numeric_parameter(block.parameters, "t0", 0.0),
                )
            )
            continue
        if block.type in SINK_TYPES:
            label = str(block.parameters.get("label") or "").strip() or block.id
            reference = (
                get_numeric_parameter(block.parameters, "reference", 0.0)
```

```
                if "reference" in block.parameters
                else None
            )
            scope_index[block.id] = len(scopes)
            scopes.append(ScopeOutput(block.id, label, reference))
            continue

        realization = block_realization(block)
        if realization.b.shape[1] != len(block.input_ports):
            raise ModelAssemblyError(
                [f"Блок '{block.id}': число входов не совпадает с моделью звена."]
            )
        output_index[block.id] = len(dynamic)
        input_offset[block.id] = total_inputs
        total_inputs += len(block.input_ports)
        for local in range(realization.order):
            label = block.id if realization.order == 1 else f"{block.id}.x{local + 1}"
            states.append(StateEntry(block.id, block.type, local, label))
        dynamic.append((block, realization))

    realizations = [realization for _, realization in dynamic]
    n = sum(r.order for r in realizations)
    n_out = len(dynamic)
    n_src = len(sources)
    p = len(scopes)

    a_d = _block_diagonal([r.a for r in realizations], n, n)
    b_d = _block_diagonal([r.b for r in realizations], n, total_inputs)
    c_d = _block_diagonal([r.c for r in realizations], n_out, n)
    d_d = _block_diagonal([r.d for r in realizations], n_out, total_inputs)
    x0 = np.concatenate([r.x0 for r in realizations]) if realizations else np.zeros(0)

    m = np.zeros((total_inputs, n_out))
    nr = np.zeros((total_inputs, n_src))
    s_y = np.zeros((p, n_out))
    s_r = np.zeros((p, n_src))
    blocks_by_id = {block.id: block for block in flat.blocks}

    for connection in flat.connections:
        target = blocks_by_id[connection.to_block]
        if target.type in SINK_TYPES:
            row_matrix_y, row_matrix_r, row = s_y, s_r, scope_index[target.id]
        else:
            row = input_offset[target.id] +
target.input_ports.index(connection.to_port)
            row_matrix_y, row_matrix_r = m, nr
        if connection.from_block in source_index:
            row_matrix_r[row, source_index[connection.from_block]] = 1.0
        else:
            row_matrix_y[row, output_index[connection.from_block]] = 1.0

    g = d_d @ m
    owners = [block.id for block, _ in dynamic]
    y_solution, loops = _solve_algebraic_equations(g, np.hstack([c_d, d_d @ nr]),
owners)
    y_x, y_r = y_solution[:, :n], y_solution[:, n:]

    return LinearModel(
        a=a_d + b_d @ m @ y_x,
        b=b_d @ (m @ y_r + nr),
        c=s_y @ y_x,
        d=s_y @ y_r + s_r,
        x0=x0,
        states=states,
        sources=sources,
        scopes=scopes,
        algebraic_loops=loops,
    )
```

Приложение Б. Расчёт ЭГСП в MATLAB/Octave

Скрипт сворачивает структурную схему ЭГСП (раздел 3.3) и считает переходный процесс; результаты сравниваются с программой в главе 5.

Листинг 2 – Независимый расчёт ЭГСП (backend/verification/egsp.m)

```
% Независимая проверка ЭГСП 8-го порядка (MATLAB / GNU Octave, пакет control).
% Передаточная функция строится последовательным сворачиванием структурной
% схемы, как в практическом занятии 02 (рис. 1, приложение 1), но численно.
% Результат пишется в docs/study/octave_egsp.csv для сравнения с программой.
if exist('OCTAVE_VERSION', 'builtin'), pkg load control; end
s = tf('s');
k1 = 50;  a1 = 0.4;  a0 = 80;  b0 = 1;
c2 = 0.3e-6;  c1 = 7e-5;  c0 = 0.1;  d0 = 1;  k4 = 10;
k2 = 1e-3;  k3 = 1/1e-6;  g1 = 0.785e-4;
e2 = 0.005;  e1 = 0.25;  e0 = 2;  f0 = 0.785e-4;  k5 = 4.3;
h1 = 647e-6/(2*1e7);  h0 = 1e-10;  l0 = 1;  q1 = 1.99e-3;
n2 = 10;  n1 = 10;  n0 = 0;  r0 = 1.99e-3;  k6 = 0.1;

W1 = k1;  W2 = b0/(a1*s + a0);  W3 = d0/(c2*s^2 + c1*s + c0);
W4 = k2;  W5 = k3;  W6 = f0/(e2*s^2 + e1*s + e0);  W7 = k5;
W8 = l0/(h1*s + h0);  W9 = r0/(n2*s^2 + n1*s + n0);
W10 = q1*s;  W11 = g1*s;  W12 = k4;  W13 = k6;

W01 = feedback(W5*W6, W11);              % золотник с гидродинамической обратной
связью
W02 = feedback(W3*W4*W01, W12);          % якорь ЭМП с механической обратной связью
W03 = feedback(W8*W9, W10);              % гидроцилиндр с поршнем
W   = minreal(feedback(W1*W2*W02*W7*W03, W13), 1e-9);   % главный контур

[num, den] = tfdata(W, 'v');
k = find(abs(den) > 0, 1);  num = num(k:end)/den(k);  den = den/den(k);
num = num(find(abs(num) > 0, 1):end);
t = (0:0.0005:0.7)';
y = lsim(W, 0.01*ones(size(t)), t);
write_result('../../docs/study/octave_egsp.csv', num, den, pole(W), t, y);
```